

Chapter 4

Processes and Virtualization

4.1 The Abstraction: A Process

The most fundamental abstraction that the operating system provides to users is the process. Informally, a process is simply a running program. At any point in time, the program itself is just a lifeless thing: it sits on the disk, a bunch of instructions and perhaps static data, waiting to be executed. It is the operating system that takes these bytes and gets them running.

To provide the illusion of multiple CPUs, the operating system creates the process abstraction. By running one process, then stopping it and running another, and so forth, the OS promotes the illusion that many virtual CPUs exist when in fact only one or a few physical CPUs are actually present. This basic technique is known as time sharing of the CPU.

Time sharing allows users to run as many concurrent processes as they would like; the potential cost is performance, as each will run more slowly if the CPU must be shared. To implement virtualization of the CPU, the OS needs both some low-level machinery called mechanisms, and high-level intelligence known as policies.

4.2 Process Creation: The fork() System Call

To understand how a process is created, consider the following C program that utilizes the fork() system call to create a new child process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("child process (pid:%d)\n", (int) getpid());
    } else {
        printf("parent of %d (pid:%d)\n", rc, (int) getpid());
    }
    return 0;
}
```

When this program runs, the operating system creates a nearly exact copy of the calling process. Notice that the child process does not start at main; rather, it returns from fork as if it had called fork itself.

The return value from fork is different in the parent and child: the child receives 0, while the parent receives the PID of the newly created child.

4.3 Process States and Transitions

In a simplified view, a process can be in one of three states: Running, Ready, or Blocked. The transitions between these states govern process scheduling.

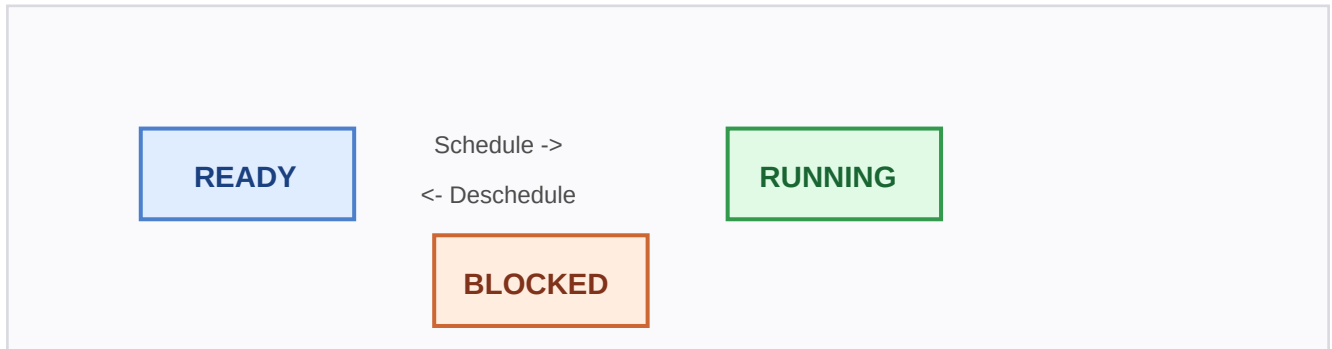


Figure 4.1: Simplified Process State Transitions

As shown in the figure above, a process moves from Running to Blocked when waiting for an I/O request to complete. Once the request finishes, an interrupt moves it back to the Ready queue.

4.4 CPU Scheduling Metrics

To evaluate scheduling policies, we define turnaround time as the time at which the job completes minus the time at which the job arrived in the system:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}} \text{ (Equation 4.1)}$$

Another metric of interest is fairness, often formalized through Jains Fairness Index which measures allocation across n jobs:

$$J(x_1, x_2, \dots, x_n) = (\text{sum}(x_i))^2 / (n * \text{sum}(x_i^2)) \text{ (Equation 4.2)}$$

Tradeoffs between turnaround time and fairness are central to operating system scheduler design. Optimizing one often comes at the expense of the other.